

Coding & Solution

Date	01 july 2026
Team ID	SWTID-2026-2152
Project Name	Emotion Detection & Learning Support Engine
Maximum Marks	5 Marks

Coding & Solution

This section evaluates the quality and completeness of your implemented code and the overall solution. Your submission should demonstrate working functionality, clean code practices, and alignment with your defined requirements.

Instructions:

1. Ensure your code is well-structured, commented, and follows best practices for your chosen technology stack.
2. The solution should address the core problem statement and implement the key functional requirements.
3. Include all source files, configuration files, and setup instructions needed to run the application.
4. Attach or link to your code repository (e.g., GitHub) in the table below.

Solution Summary

Field	Details
Field	Details
Repository Link / URL	https://github.com/Amruthav421/Emotion-Detection-Learning-Support-Engine
Programming Language(s)	Python
Framework(s)) Used	OpenCV, TensorFlow/Keras, Flask (or your specific UI framework)
Key Features Implemented	Real-time facial emotion detection, student engagement analysis, and personalized learning resource recommendations.
Pending / Incomplete Features	[List any features you are still working on, e.g., "Integration with specific LMS APIs" or "None"]

Code Quality Checklist

S.No	Code Quality Parameter	Description	Followed	Remarks
1	Consistent Indentation	Use an IDE with automatic formatter (Black/autopep8).	Yes	Cleaned via Black
2	Proper File Structure	Follows /src, /data, /models, /notebooks structure.	Yes	Organized
3	Meaningful Variable Names	Uses input_frame, predicted_emotion_label, etc.	Yes	Descriptive
4	Function / Method Names	Uses detect_emotion(), get_student_feedback(), etc.	Yes	Action-based
5	Code Comments	Comments explain the <i>why</i> of the logic.	Yes	Documented
6	Modular Design	Separate files for architecture, capture, and UI.	Yes	Decoupled
7	No Redundant Code	Constants centralized in config.py.	Yes	Optimized

Additional Notes / Comments:

The project architecture is designed for scalability and maintainability, adhering to modular programming principles. Core functionalities—including real-time facial feature extraction, emotion classification, and personalized learning recommendations—are separated into distinct modules to ensure independent testing and development. Comprehensive error handling has been integrated into the data pipeline and UI components to manage potential runtime exceptions, such as camera access failures or model inference errors, ensuring the application remains stable during use.